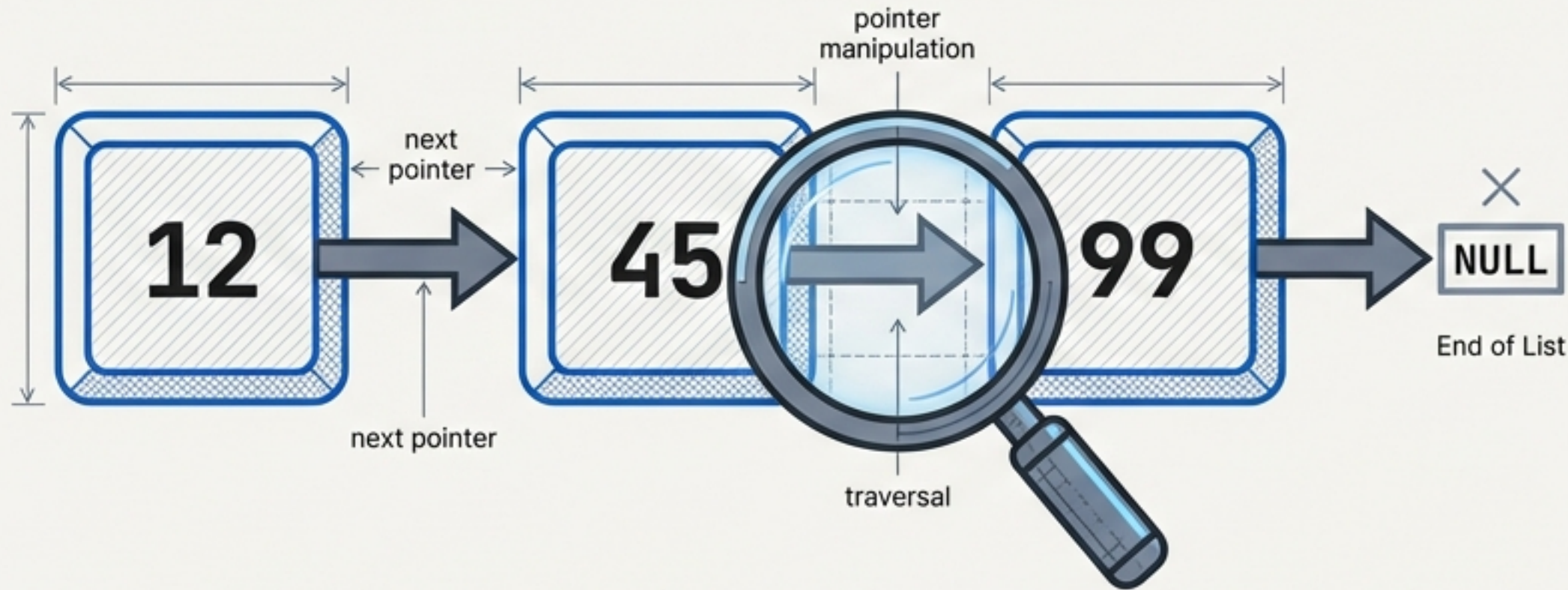


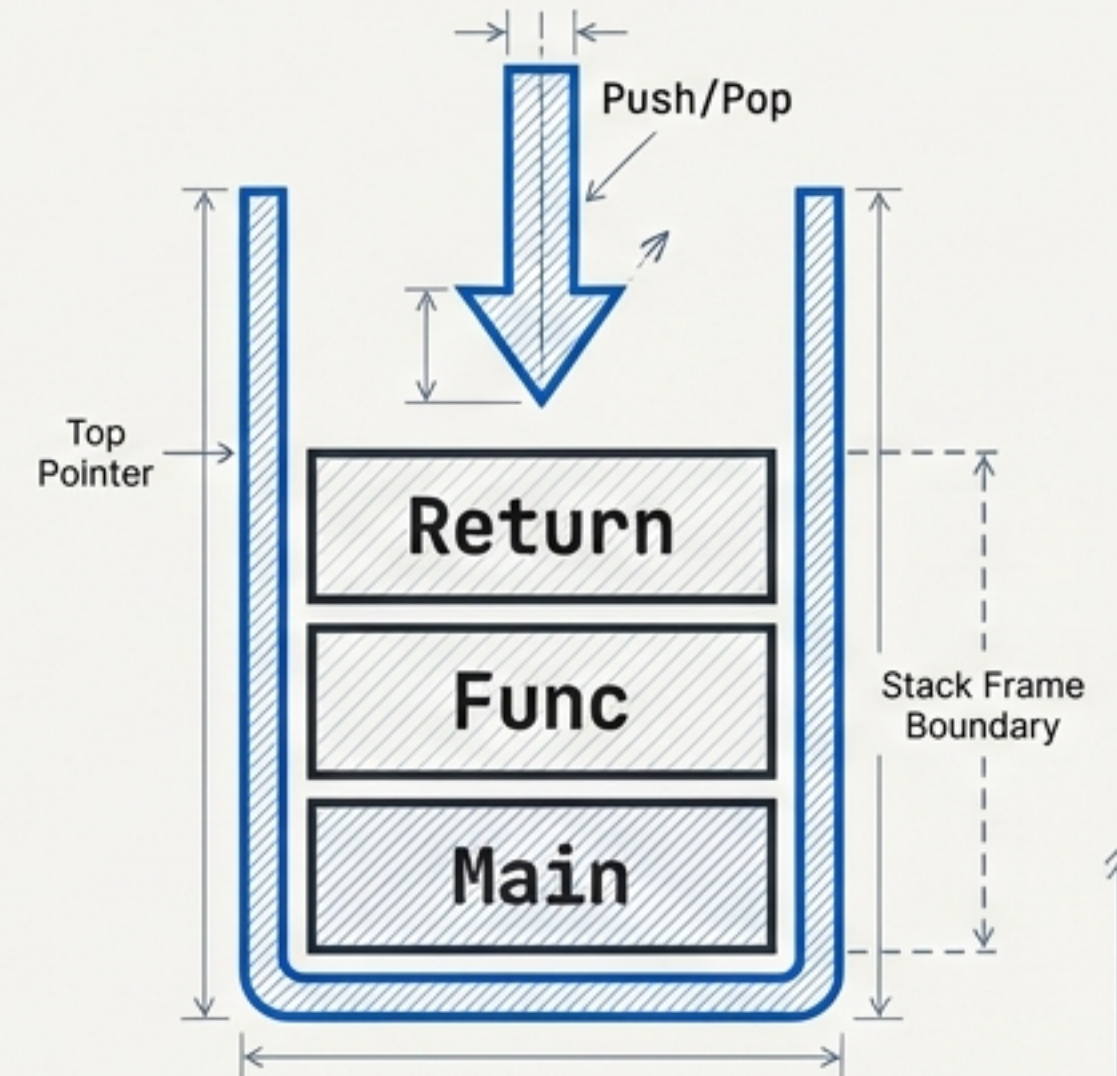
Advanced DSA Bootcamp: Visualizing Memory

Mastering Linked Lists & Stacks through Dry Running

→ LINKED LIST STRUCTURE



← STACK DATA STRUCTURE



Modern Engineering Blueprint

16:9 aspect ratio - #F9F9F9



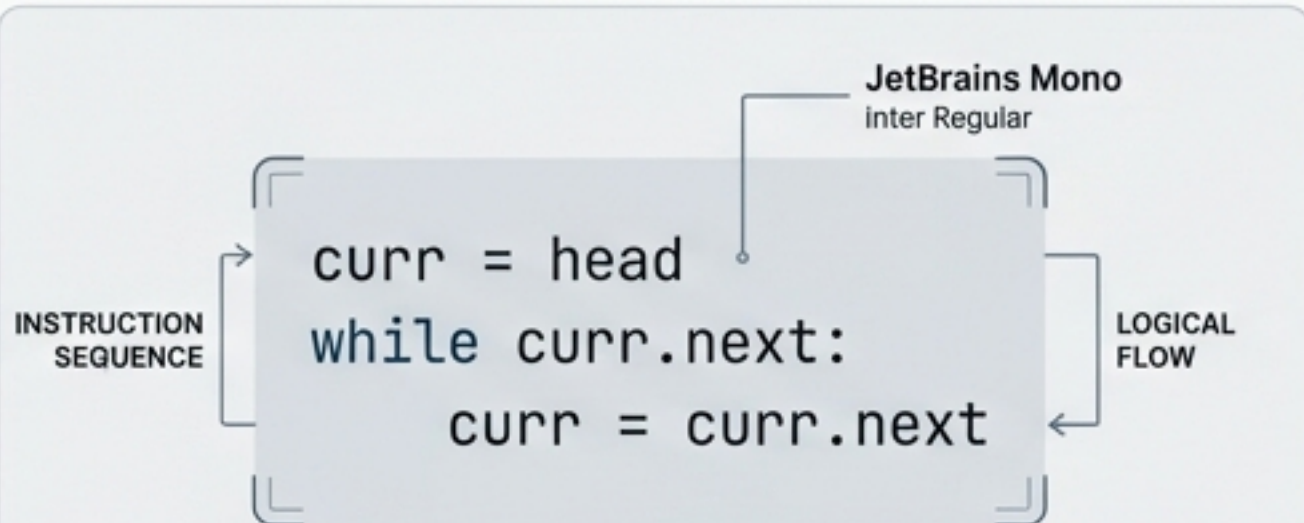
THE ABSTRACT

Inter Tight Bold, Deep Charcoal - subtitle mean the headline



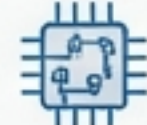
The Code

TITLE BLOCK



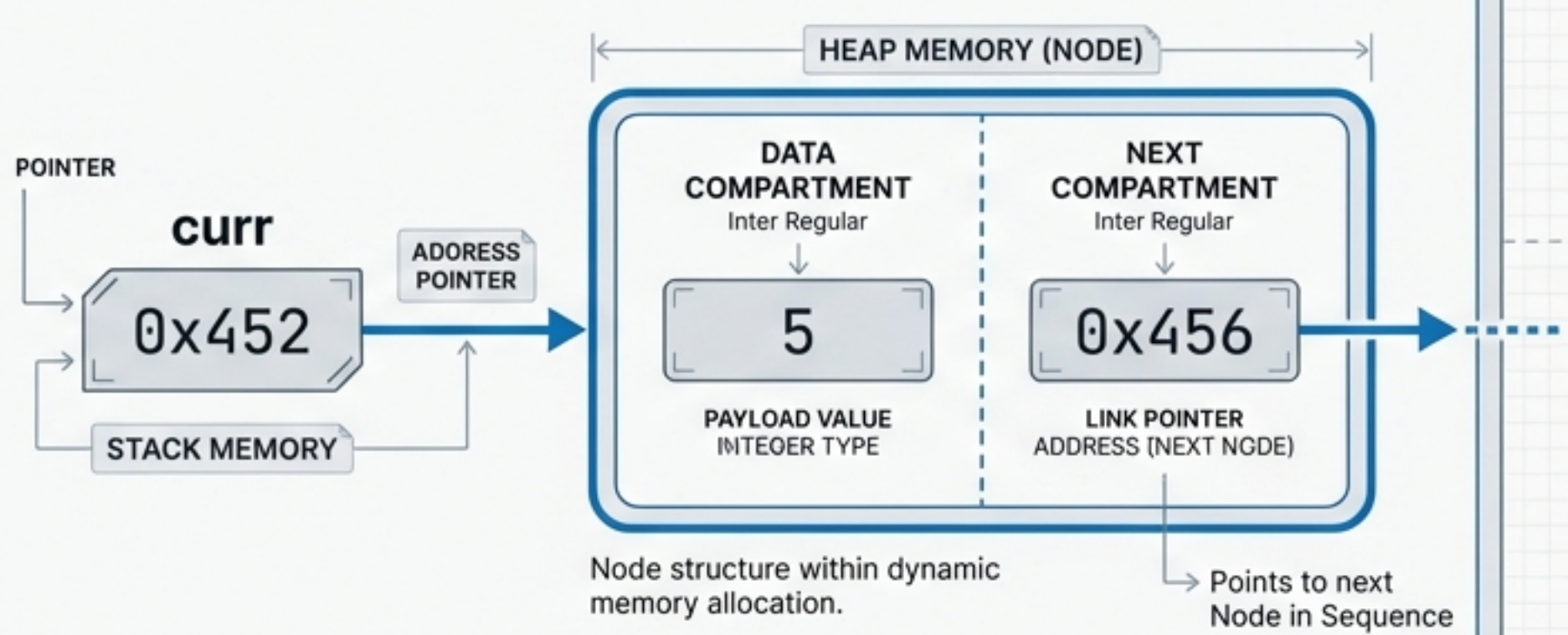
THE CONCRETE

Inter Tight Bold, Deep Charcoal - subtitle to s outritet



The Memory

TITLE BLOCK



To solve complex problems, we simulate the machine. We track **Values** (Data) and **Addresses** (The Links).

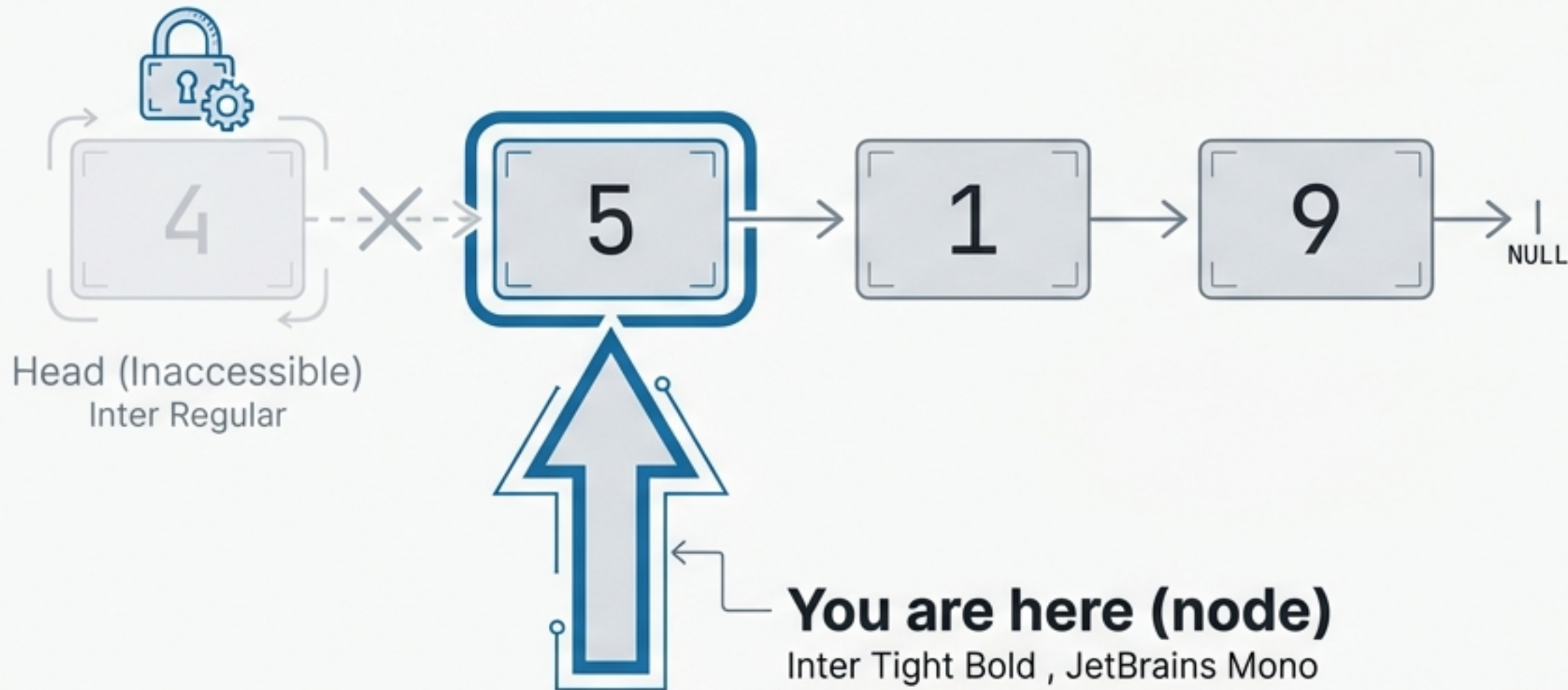
“By deleting a node, we do not mean removing it from memory entirely; we mean removing the link so it is no longer accessible.”

PROJECT: **MEMORY REPRESENTATION**

DRAWN BY: ENGINEERING DESK

Problem 1: Delete Node (No Head Access)

The constraints define the solution.



The Dilemma:

In a Singly Linked List, **you cannot traverse backward.**

You cannot access the previous node to change its pointer.

You are parachuted into the middle.

The Solution: Value Copying & Bypassing

If you can't leave, become someone else.

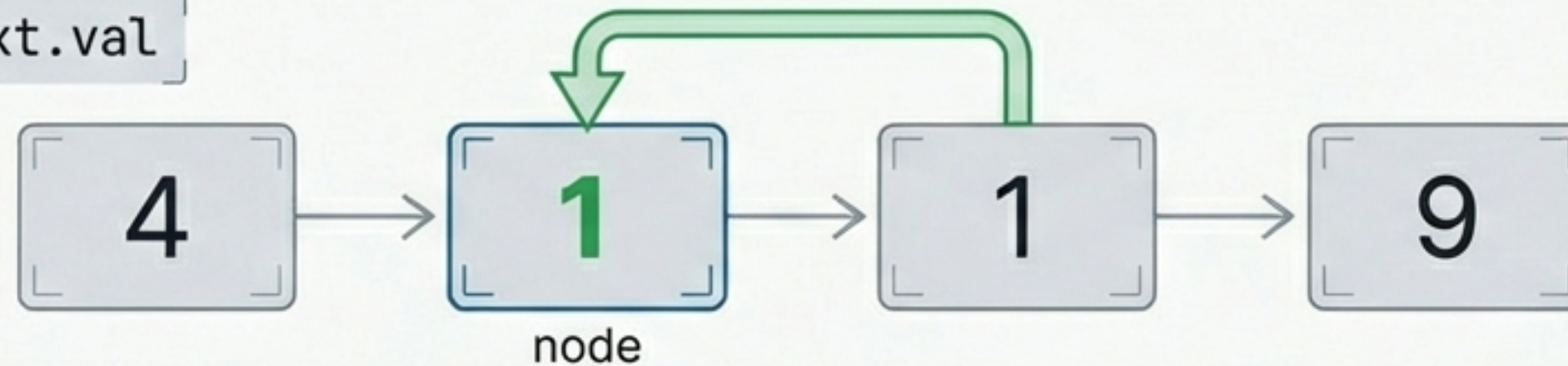


Step 1: Copy Value

JetBrains Mono

```
node.val = node.next.val
```

TITLE BLOCK

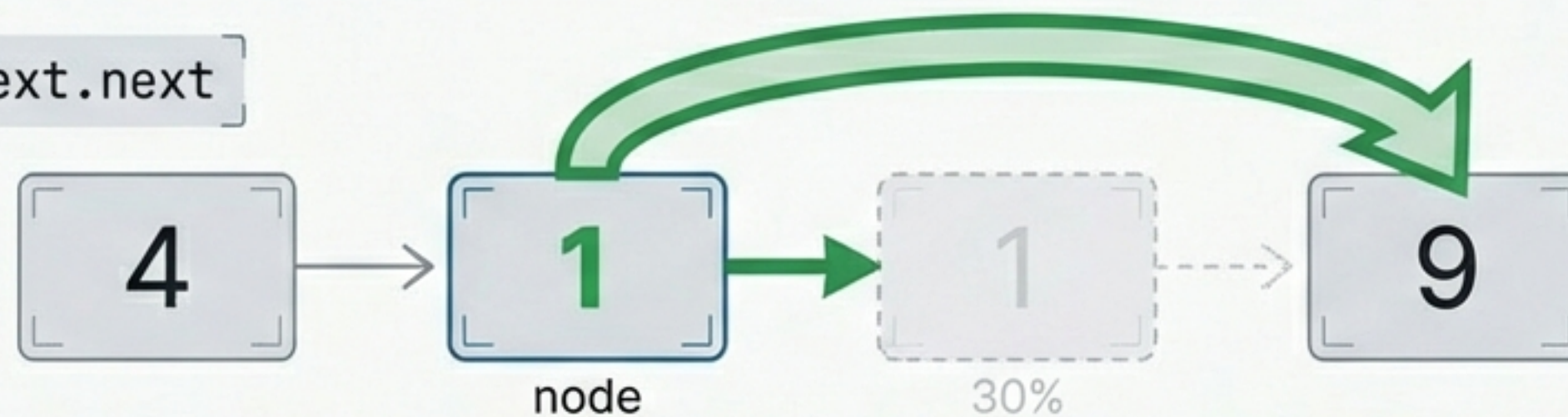


Step 2: Bypass Link

JetBrains Mono

```
node.next = node.next.next
```

TITLE BLOCK



PROJECT: **LINKED LIST OPERATIONS**

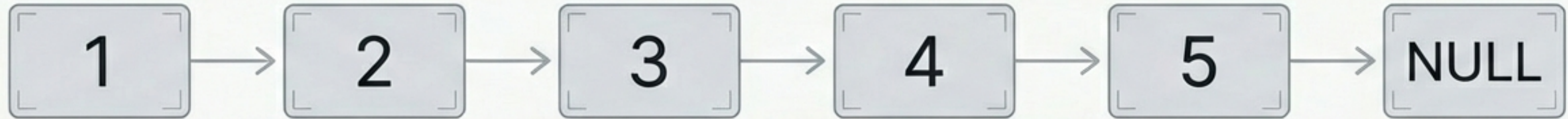
DRAWN BY: ENGINEERING DEPT. DATE: FEY.

Problem 2: Reverse Linked List

The Challenge of Breaking Chains.

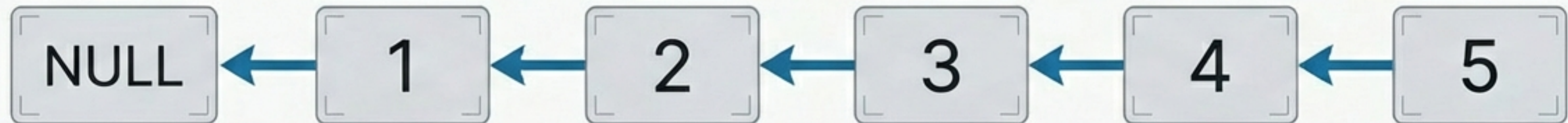


Input



Danger: If you simply turn Node 2 to point at Node 1, you sever the connection to Node 3. The rest of the list is lost in memory.

Target Output



The Mechanics: Three Pointer Strategy

We need guards to hold the structure together.



Prev (None)

The new destination.



NULL

Curr

The node we are flipping.



1

Temp (Next)

The scout. Saves the future list.



2

3

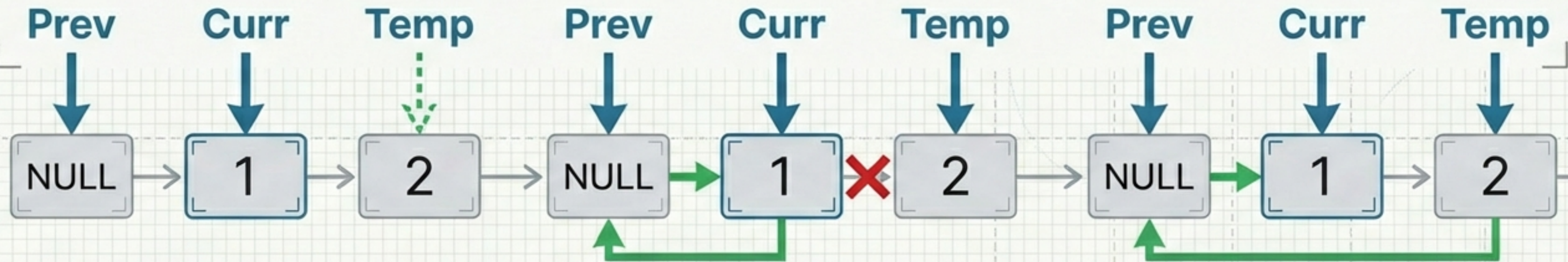
prev = None
curr = head

Dry Run: Inside the Reversal Loop

A step-by-step visualization of a single iteration.



Save → **Reverse** → **Advance**



`temp = curr.next`
(Save the rest of the list)

`curr.next = prev`
(Reverse the link)

`prev = curr; curr = temp`
(Move guards forward)

Linked List Implementation (Python)



Reverse List

```
while curr:
    temp = curr.next # 1. Save Future
    curr.next = prev # 2. Reverse Link
    prev = curr      # 3. Move Prev
    curr = temp      # 4. Move Curr
return prev
```

Prev is the new Head

Delete Node

```
# Overwrite value
node.val = node.next.val
# Bypass link
node.next = node.next.next
```

O(1) Space - No new data structures created.

Problem 3: Removing Stars From a String

Processing Data LIFO (Last-In, First-Out)

Inter Regular Slate Grey

leet**cod*e

Typed

The diagram illustrates the process of removing characters from the string "leet**cod*e". The string is shown with red asterisks representing backspaces. Red arrows indicate the removal of characters: one arrow points from the 'e' in "leet" to the first asterisk, and another points from the 'd' in "cod" to the second asterisk. A bracket under "leet" is labeled "Typed".

Result: lecoe

Rule: A Star (*) acts as a backspace. It removes the immediate left non-star character.

We only care about the most recently added character.

PROJECT: STRING OPERATIONS

Inter Tight Bold

DRAWN BY: ENGINEERING DEPT.

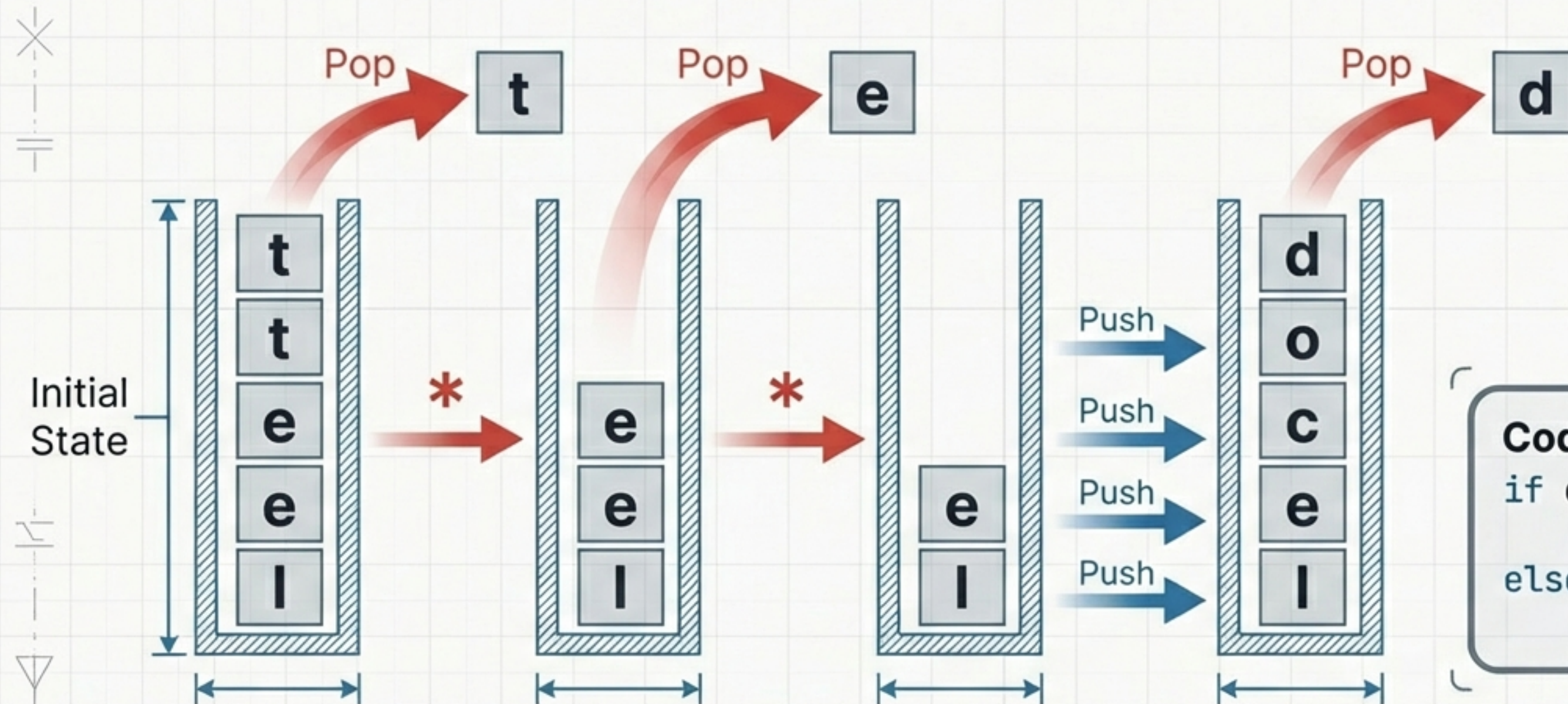
Inter Regular

DME PEV

The Stack as a Container

Visualizing the Push and Pop operations

Inter Regular Deep Charcoal



Code Logic Box

```
if char == '*':  
    stack.pop()  
else:  
    stack.append(char)
```

PROJECT: STACK OPERATIONS

Inter Tight Bold

DRAWN BY: ENGINEERING DEPT.

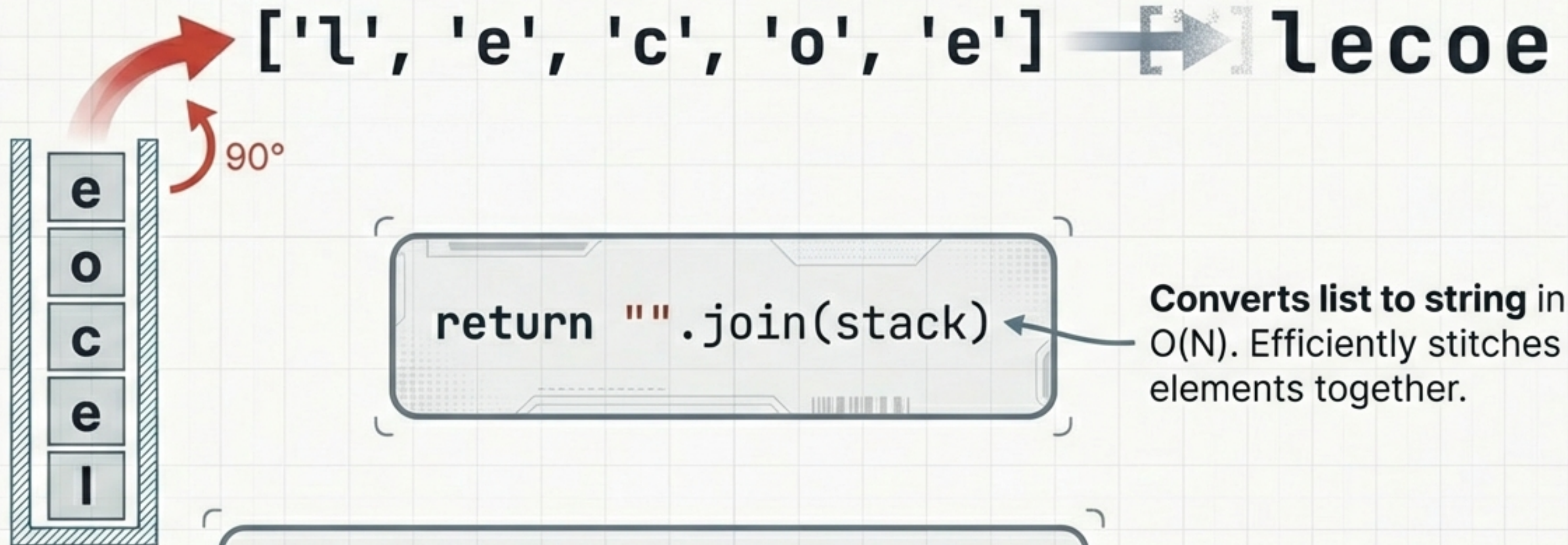
Inter Regular

DME PEV

Constructing the Result

Optimization & Edge Cases

Inter Regular Slate Grey



Insight

The operation is always possible. We don't need to check for empty stack errors on valid inputs.

PROJECT: STRING OPERATIONS

Inter Tight Bold

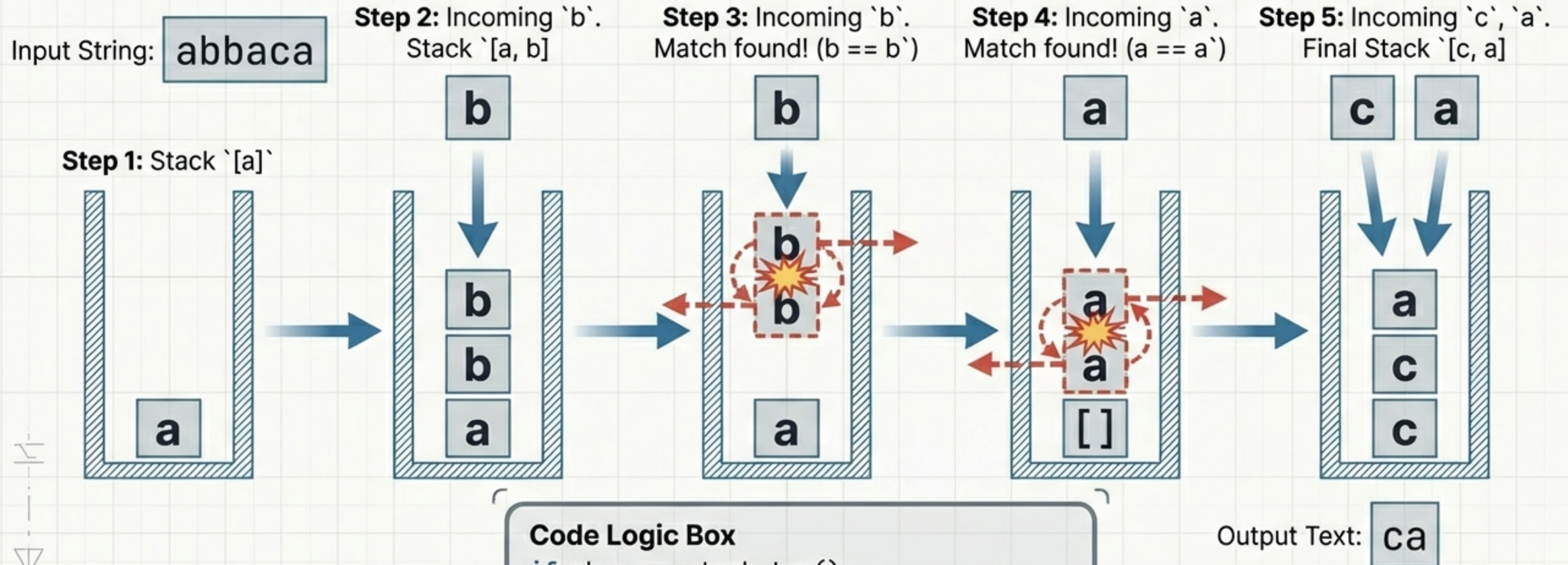
DRAWN BY: ENGINEERING DEPT.

Inter Regular

DME PEV

Pattern Variation: Adjacent Duplicates

The 'Match-3' Logic



PROJECT: PATTERN MATCHING

Inter Tight Bold

DRAWN BY: ENGINEERING DEPT.

Inter Regular

DME PEV

Stack Logic in Code

Inter Tight Bold, Deep Charcoal

Input: "leet**code"

```
stack = []
for char in s:
    if char == '*':
        stack.pop()
    else:
        stack.append(char)
```

Input: "abbaca"

```
stack = []
for char in s:
    # Check top of stack
    if stack and stack[-1] == char:
        stack.pop()
    else:
        stack.append(char)
```

Peek at the top element without removing it.
Inter Regular

PROJECT: STACK ALGORITHMS

Inter Tight Bold

DRAWN BY: ENGINEERING DEPT.

Inter Regular

DME PEV

Bootcamp Takeaways

Linked Lists



Mental Model: Chains & Links.

Use multiple pointers (Prev, Curr, Temp) to manipulate structure without losing access.

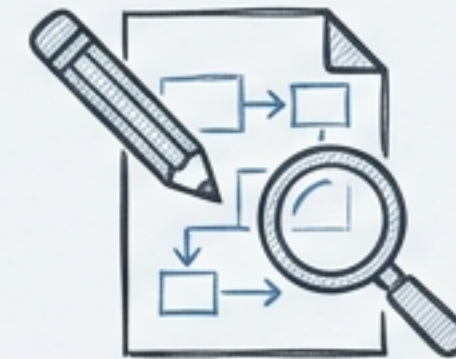
Stacks



Mental Model: Vertical Pile.

Perfect for LIFO operations, "Undo" features, and finding nearest neighbors.

Interview Tip



The Dry Run.

Don't guess. Draw the boxes and arrows. If you can visualize the memory change, the code writes itself.